

Trace 的详细分析以及功能介绍

SDSoC 开发环境

SDSoC 是 Xilinx 旗下，用于设计开发 Zynq SoC 和 MPSoC 异构嵌入式系统的基于 Eclipse 的集成开发环境，可以完成从 C/C++ 到指定目标平台上功能完整的硬件/软件系统的编译、实现、调试执行等全过程。

由 SDSoC 产生的系统是高性能的，同时也是复杂的硬件/软件协同工作系统。要了解在这样的系统中应用程序的执行过程是比较困难的，因为你需要同时关注在处理器上运行的软件部分和在硬件可编程逻辑上执行的加速部分，以及数据在二者之间的传递过程和顺序等。

Trace 事件追踪功能

2015.4 版的 SDSoC 中新增了一个功能，叫做 Trace，即事件追踪功能，可以帮助设计开发人员深入了解应用程序的执行过程中整个系统上发生了哪些事件以及发生的顺序，用户可以以此为依据进行系统优化。

Trace 提供的是一个详细视图，用户可以在其中观察应用，硬件/软件分区

和系统设计选择的性能，它提供的是一个以时间轴为单位而量化的事件追踪，包括在处理器上运行的软件事件，在可编程逻辑上运行的硬件加速事件和在系统中的数据传输链路事件。

由于需要读取上板应用的数据，要启动事件跟踪，用户必须将开发板经由 JTAG 连接到主机。

在 SDSoC 中启动 Trace 后，事件追踪功能就会被自动插入到软件代码和硬件 IP 中。然后，硬件部分（包括完成事件追踪功能的监控 IP）经过综合和布局布线，产生比特流，而软件跟踪代码则被编译到普通用户应用中。硬件和软件的事件追踪时间戳被收集到了一个追踪流中，并且以在可编程架构中缓存的方式记录下来，由主机通过 JTAG 访问并读回数据。

下图橙色高亮的部分，就是应用 Trace 功能后 SDSoC 自动插入的监控 IP，用以完成和记录事件追踪功能。

收集事件追踪数据的接口根据不同的操作系统而略有不同。对于 Standalone 和 FreeRTOS 操作系统，用户程序 ELF 使用 USB / JTAG 接口下载到电路板上进行配置和调试，断点也是经由了同样的 USB / JTAG 接口而插入，事件跟踪的数据采集，也是在相同的 USB / JTAG 接口上读取。

对于 Linux，一旦操作系统经由 SD 卡启动，ELF 已经复制后，便可以经过 TCP/TCF 运行调试，断点也可以经由 TCP/TCP 插入，但事件跟踪的数据仍然需要通过 USB / JTAG 接口读出。换句话说，要在 Linux 应用程序上启动事件追踪，USB / JTAG 和 TCP / TCF 接口都需要。

Trace 视图

下图所示就是 SDSoC GUI 提供的 Trace 视图。用户应用程序的每个跟踪事件被赋予一个唯一的轨迹名称，并且对应到同一个时间轴。通常，一个跟踪点可以通过应用程序的执行创建多个轨迹，例如，如果相同的代码被循环执行，或者在加速器中被多次调用。

每个事件追踪轨迹都有几个不同的属性：名称，类型，以及启动，停止和持续时间。如果将鼠标悬停在某个轨迹上，就可以看到这个轨迹的详细属性。Trace 视图中以不同的颜色区别不同类型的轨迹，例如橙色代表软件事件追踪，绿色代表硬件事件而蓝色代表数据的交互。

关于 Trace 更详细的介绍和分步骤教程，请关注 SDSoC 安装目录下的 docs 目录，其中有一个 SDSoC_Trace2015.4.pdf 文档可供参考。

由于 Trace 是在 SDSoC 2015.4 版首次发布，且目前还处于 Beta 阶段，其仅能完成有限的事件追踪功能，我们将在以后的版本中不断完善，提供更全面的追踪功能。敬请关注。如果您想要立即试用 SDSoC 2015.4 和 Trace 功能，请联系当地的 Xilinx 销售代表。